

# DP-LMI package

## User Manual

Version v0.1.1

July 2026

### **Abstract**

This manual describes the current public behavior of the DP-LMI package. The package uses `dpbase` as the backend parent for cell-local Bernstein coefficient storage, provides `dpmat` for known finite real matrix functions on tensor grids, provides `dpvar` for YALMIP-backed Bernstein decision expressions and `rhodiff` derivatives, and provides `dplmi` for direct coefficient-wise YALMIP constraint assembly. The manual is written as a reference manual: conceptual Bernstein material comes first, and each class chapter then documents supported call forms, arguments, outputs, examples, and limits.

# Contents

|          |                                                                                        |           |
|----------|----------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Bernstein And Grid Background</b>                                                   | <b>4</b>  |
| 1.1      | Local Coordinates And Degree-Two Coefficients . . . . .                                | 4         |
| 1.2      | Multiplication, Degree Growth, And Convolution . . . . .                               | 4         |
| 1.3      | Degree Elevation And Polynomial Representation . . . . .                               | 5         |
| 1.4      | Storage And Hypercube Traversal . . . . .                                              | 5         |
| <b>2</b> | <b>Installation, Verification, And Lookup</b>                                          | <b>7</b>  |
| 2.1      | Installation . . . . .                                                                 | 7         |
| 2.2      | Verification . . . . .                                                                 | 7         |
| 2.3      | Quick Start . . . . .                                                                  | 7         |
| 2.4      | Global Function Lookup . . . . .                                                       | 8         |
| <b>3</b> | <b>dpbase Backend Reference</b>                                                        | <b>9</b>  |
| 3.1      | Lookup . . . . .                                                                       | 9         |
| 3.2      | dpbase Constructor . . . . .                                                           | 9         |
| 3.3      | Storage Fields . . . . .                                                               | 10        |
| 3.4      | Inspection: <code>cells</code> , <code>lbls</code> , And <code>coeffs</code> . . . . . | 10        |
| 3.5      | Counts And Shape . . . . .                                                             | 11        |
| <b>4</b> | <b>dpmat Reference</b>                                                                 | <b>13</b> |
| 4.1      | Lookup . . . . .                                                                       | 13        |
| 4.2      | Construction . . . . .                                                                 | 13        |
| 4.3      | <code>evaluate</code> . . . . .                                                        | 15        |
| 4.4      | Visualization And Inspection . . . . .                                                 | 16        |
| 4.4.1    | <code>bernsteinTable</code> . . . . .                                                  | 16        |
| 4.4.2    | <code>disp</code> And <code>display</code> . . . . .                                   | 17        |
| 4.4.3    | <code>plot</code> . . . . .                                                            | 18        |
| 4.5      | Algebra Operators . . . . .                                                            | 20        |
| 4.6      | Matrix Methods . . . . .                                                               | 21        |
| 4.7      | Indexing And Assignment . . . . .                                                      | 23        |
| 4.8      | Limits . . . . .                                                                       | 24        |
| <b>5</b> | <b>dpvar Reference</b>                                                                 | <b>25</b> |
| 5.1      | Lookup . . . . .                                                                       | 25        |
| 5.2      | Construction . . . . .                                                                 | 25        |
| 5.3      | <code>rhodiff</code> . . . . .                                                         | 27        |
| 5.4      | Affine And Mixed Algebra . . . . .                                                     | 29        |
| 5.5      | Matrix Methods . . . . .                                                               | 30        |
| 5.6      | Indexing And Assignment . . . . .                                                      | 32        |
| 5.7      | Comparisons To <code>dplmi</code> . . . . .                                            | 33        |
| 5.8      | Limits . . . . .                                                                       | 33        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>6</b> | <b>dplmi Reference</b>                          | <b>35</b> |
| 6.1      | Lookup . . . . .                                | 35        |
| 6.2      | Construction And Comparisons . . . . .          | 35        |
| 6.3      | Direct Coefficient Assembly . . . . .           | 36        |
| 6.4      | toYalmip . . . . .                              | 36        |
| 6.5      | Solver-Facing Use . . . . .                     | 37        |
| <b>7</b> | <b>Examples</b>                                 | <b>38</b> |
| 7.1      | Rate-Dependent Block LMI Solver Smoke . . . . . | 38        |
| <b>8</b> | <b>Current Limits</b>                           | <b>40</b> |

# 1

## Bernstein And Grid Background

### 1.1 Local Coordinates And Degree-Two Coefficients

On one physical interval  $[\rho_i, \rho_{i+1}]$ , the package uses the local coordinate

$$a = \frac{\rho_{i+1} - \rho}{\rho_{i+1} - \rho_i}.$$

Thus  $a = 1$  at the lower endpoint and  $a = 0$  at the upper endpoint. A degree- $m$  Bernstein polynomial on that cell is

$$P(\rho) = \sum_{j=0}^m \binom{m}{j} a^{m-j} (1-a)^j C_j.$$

The degree-two case is the first case where the middle Bernstein scale matters:

$$P(\rho) = a^2 C_0 + 2a(1-a)C_1 + (1-a)^2 C_2.$$

The package stores the coefficient matrices  $C_0, C_1, C_2$ , not sampled values of the expanded monomial form. For matrix-valued objects, each  $C_j$  is a finite real matrix for `dpmat`, or a YALMIP affine matrix expression for `dpvar`.

For tensor grids, a local Bernstein coefficient label is a row vector. For example, a two-parameter degree-one cell has labels  $[0, 0]$ ,  $[0, 1]$ ,  $[1, 0]$ , and  $[1, 1]$ . Each label selects one tensor-product Bernstein basis term, one local index per parameter direction.

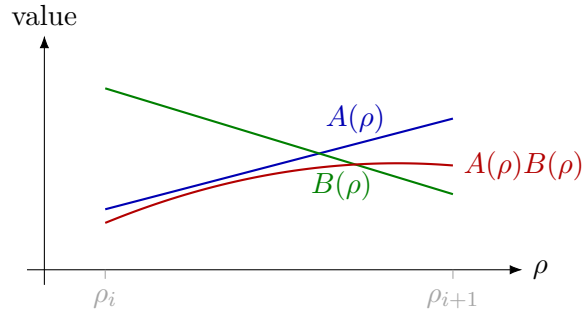


Figure 1: Local Bernstein multiplication raises polynomial degree.

### 1.2 Multiplication, Degree Growth, And Convolution

Multiplication raises Bernstein degree. In one parameter, multiplying two degree-one factors gives a degree-two product:

$$(aA_0 + (1-a)A_1)(aB_0 + (1-a)B_1) = a^2 A_0 B_0 + a(1-a)(A_0 B_1 + A_1 B_0) + (1-a)^2 A_1 B_1.$$

Because the degree-two Bernstein middle basis is  $2a(1-a)$ , the stored middle coefficient is  $(A_0 B_1 + A_1 B_0)/2$ .

This is not a special rule for linear factors. If  $B_i^m(a)$  denotes the  $i$ -th Bernstein basis function of degree  $m$ , then

$$B_i^m(a)B_j^n(a) = \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{i+j}} B_{i+j}^{m+n}(a).$$

For coefficient arrays this is a convolution over coefficient labels. In one parameter,

$$C_k = \sum_{\substack{i+j=k \\ 0 \leq i \leq m, 0 \leq j \leq n}} A_i B_j \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}}, \quad k = 0, \dots, m+n.$$

In  $\ell$  parameters, replace the scalar labels  $i, j, k$  by multi-indices  $\alpha, \beta, \gamma$ , use  $\alpha + \beta = \gamma$ , and multiply the binomial scale over parameter directions:

$$C_\gamma = \sum_{\alpha+\beta=\gamma} A_\alpha B_\beta \prod_{r=1}^{\ell} \frac{\binom{m}{\alpha_r}\binom{n}{\beta_r}}{\binom{m+n}{\gamma_r}}.$$

This is the coefficient rule used by `dpmat * dpmat` and by supported known-data products involving `dpvar`.

### 1.3 Degree Elevation And Polynomial Representation

A polynomial represented in Bernstein degree  $m$  can also be represented in any higher degree  $M \geq m$ . In one parameter, the elevated coefficient  $\hat{C}_k$  of degree  $M$  is

$$\hat{C}_k = \sum_{i=\max(0, k-(M-m))}^{\min(m, k)} C_i \frac{\binom{m}{i}\binom{M-m}{k-i}}{\binom{M}{k}}, \quad k = 0, \dots, M.$$

Tensor grids apply the same formula by tensor product over each parameter direction. The package uses degree elevation when two coefficient-backed objects with the same grid bounds but different Bernstein degree must be combined by addition, subtraction, concatenation, assignment, or other coefficient-wise operations.

### 1.4 Storage And Hypercube Traversal

The physical grid and the local Bernstein labels are separate. For  $\ell$  parameters with node counts  $N_1, \dots, N_\ell$ , the number of physical hypercubes is

$$\prod_{r=1}^{\ell} (N_r - 1).$$

For Bernstein degree  $m$ , each physical hypercube stores

$$(m+1)^\ell$$

local coefficients. The helper ordering used by `cells` and `lbls` is lexicographic with earlier dimensions varying more slowly.

#### Example.

The storage counts follow directly from the physical grid and the local Bernstein degree.

```
A = dpmat({[0 1 2], [10 20]}, {1 2; 3 4; 5 6}, Degree=1);
A.ncell()
A.ncoeff()
A.npar()
```

```
ans =
     2

ans =
     4

ans =
     2
```

The same object exposes the local Bernstein labels and physical-cell traversal order used throughout the package.

```
A = dpmat({[0 1 2], [10 20]}, {1 2; 3 4; 5 6}, Degree=1);
A.lbls()
A.cells()
```

```
ans =
     0     0
     0     1
     1     0
     1     1

ans =
     1     1
     2     1
```

For a short coefficient display, `bernsteinTable(A, "oneLine")` prints the local Bernstein expression instead of every metadata column:

```
A = dpmat({[0 1]}, {1, 3}, Degree=1);
T = bernsteinTable(A, "oneLine");
disp(T)
```

| CellSubscript | Expression      |
|---------------|-----------------|
| -----         | -----           |
| {[1]}         | "a*1 + (1-a)*3" |

# 2

## Installation, Verification, And Lookup

### 2.1 Installation

Start MATLAB in the repository root, or set `projectRoot` to the absolute path of the checked-out repository. Add the package recursively, then remove the documentation directory from the executable path.

```
projectRoot = "path/to/Differential Parameter-Dependent Linear Matrix Inequality";
addpath(genpath(projectRoot));
rmpath(genpath(fullfile(projectRoot, "doc")));
```

For local work from the repository root, this package also resolves from:

```
addpath(pwd)
```

### 2.2 Verification

Run the current MATLAB test entry point from the repository root:

```
results = tests.run_all();
```

The test entry point covers helper utilities, `dpbase`, `dpmat`, `dpvar`, and `dplmi`. The YALMIP-backed `dpvar` and `dplmi` tests require YALMIP. Solver smoke tests additionally require an SDP solver.

### 2.3 Quick Start

```
A = dpmat({[0 1]}, {1, 3}, Degree=1);
T = bernsteinTable(A, "oneLine");
disp(T)
```

| CellSubscript | Expression      |
|---------------|-----------------|
| -----         | -----           |
| {[1]}         | "a*1 + (1-a)*3" |

```
yalmip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
C = P >= 0;
F = toYalmip(C);
```

F can be passed to ordinary YALMIP calls, for example:

```
sol = optimize(F, objective, opts);
```

## 2.4 Global Function Lookup

| Category           | Entries                                                                                                                                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend            | <code>dpbase</code> , <code>constructor</code> , <code>cells</code> , <code>lbls</code> , <code>coeffs</code> , <code>ncell</code> , <code>ncoeff</code> , <code>npar</code> , <code>size</code>            |
| Known data         | <code>dpmat</code> , <code>construction</code> , <code>evaluate</code> , <code>bernsteinTable</code> , <code>disp/display</code> , <code>plot</code> , <code>operators</code> , <code>matrix methods</code> |
| Decision variables | <code>dpvar</code> , <code>construction</code> , <code>rhodiff</code> , <code>affine/mixed algebra</code> , <code>matrix methods</code> , <code>comparisons</code>                                          |
| LMIs               | <code>dplmi</code> , <code>construction</code> , <code>direct coefficient assembly</code> , <code>toYalmip</code> , <code>solver examples</code>                                                            |

# 3

## dpbase Backend Reference

**dpbase** is the developer-facing backend parent for **dpmat** and **dpvar**. Ordinary users should model known data with **dpmat** and decision expressions with **dpvar**. This section documents **dpbase** because it explains the shared storage, traversal, and inspection contract used by both public modeling classes.

### 3.1 Lookup

| Task                  | Function or field                                                                |
|-----------------------|----------------------------------------------------------------------------------|
| Create backend object | <code>dpbase</code>                                                              |
| Inspect cells/labels  | <code>cells</code> , <code>lbls</code> , <code>coeffs</code>                     |
| Count sizes           | <code>ncell</code> , <code>ncoeff</code> , <code>npar</code> , <code>size</code> |
| Read storage fields   | <code>GridInfo</code> , <code>LocalValues</code> , <code>metadata</code> fields  |

### 3.2 dpbase Constructor

**Syntax.**

```
obj = dpbase(gridVectors, matrixSize, degree)
obj = dpbase(gridVectors, matrixSize, degree, localValues)
obj = dpbase(..., IsContinuous=tf)
obj = dpbase(..., ContainsDecision=tf)
obj = dpbase(..., HasRateDependence=tf)
obj = dpbase(..., RateBounds=rb)
obj = dpbase(..., SourceSummary=txt)
```

**Arguments, options, and outputs.**

- `gridVectors` is a cell vector with one strictly increasing node vector per parameter, such as  $\{[0 \ 1 \ 2], [10 \ 20]\}$ .
- `matrixSize` is a positive two-element matrix size, such as  $[2 \ 2]$ .
- `degree` is a nonnegative integer Bernstein degree, such as 1.
- `localValues` is the nested physical-cell storage. When omitted, **dpbase** creates a zero coefficient-backed object.
- `RateBounds` is empty or an  $\ell \times 2$  matrix of finite lower and upper rate bounds, such as `RateBounds=[-1 1; -2 2]`.

**Example.**

This backend example constructs a two-parameter parent object. The object has two physical cells because the first parameter has two intervals and the second parameter has one interval.

```

obj = dpbase({[0 1 2], [10 20]}, [2 2], 1);
class(obj)
obj.MatrixSize
obj.Degree
obj.ncell()
obj.ncoeff()
obj.npar()
size(obj)

```

```

ans =
    'dpbase'

ans =
     2     2

ans =
     1

ans =
     2

ans =
     4

ans =
     2

ans =
     2     2

```

The printed `ncoeff` value is  $(1 + 1)^2 = 4$ , one local Bernstein coefficient for each tensor-product label in the two parameter directions.

### 3.3 Storage Fields

| Field                          | Meaning                                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>GridInfo</code>          | Grid vectors, node counts, and grid bounds.                                                                       |
| <code>MatrixSize</code>        | Size of each matrix coefficient or matrix expression.                                                             |
| <code>Degree</code>            | Local Bernstein degree stored in every physical cell.                                                             |
| <code>LocalValues</code>       | Nested cell tree indexed by physical-cell subscript; each leaf stores coefficient cells.                          |
| <code>IsContinuous</code>      | Metadata flag; subclasses are responsible for boundary sharing if continuity is true.                             |
| <code>ContainsDecision</code>  | True when coefficient payloads include YALMIP decision variables.                                                 |
| <code>HasRateDependence</code> | True when the object carries rate metadata or rate-vertex rows.                                                   |
| <code>RateBounds</code>        | Parameter-rate lower and upper bounds used by <code>rhodiff</code> and <code>dplmi</code> .                       |
| <code>SourceSummary</code>     | Short origin label such as <code>coefficient-backed</code> , <code>decision</code> , or <code>derivative</code> . |

### 3.4 Inspection: `cells`, `lbls`, And `coeffs`

**Syntax.**

```

subs = cells(obj)
subs = obj.cells()

labels = lbls(obj)
labels = obj.lbls()

c = coeffs(obj, cellSubs)
c = obj.coeffs(cellSubs)

```

### Arguments, options, and outputs.

`cells` returns physical hypercube subscripts in the shared traversal order. `lbls` returns local Bernstein labels in the shared coefficient order. `coeffs(obj, cellSubs)` returns the flat coefficient cell array for one physical cell; `cellSubs` may be a numeric row vector such as `[2 1]` or a cell array of scalar subscripts such as `\{2,1\}`.

### Example.

The label order is shared by `dpmat`, `dpvar`, and `dplmi`. In this two-parameter degree-one object, each physical cell stores four coefficient entries with labels `[0,0]`, `[0,1]`, `[1,0]`, and `[1,1]`.

```

obj = dpbase({[0 1 2], [10 20]}, [1 1], 1);
lbls(obj)
cells(obj)
c = coeffs(obj, [2 1]);
numel(c)

```

```

ans =
     0     0
     0     1
     1     0
     1     1

ans =
     1     1
     2     1

ans =
     4

```

## 3.5 Counts And Shape

### Syntax.

```

n = ncell(obj)
n = obj.ncell()

n = ncoeff(obj)
n = obj.ncoeff()

n = npar(obj)
n = obj.npar()

sz = size(obj)
d = size(obj, dim)

```

**Arguments, options, and outputs.**

`ncell` returns  $\prod_r (N_r - 1)$ . `ncoeff` returns  $(m + 1)^\ell$ . `npar` returns the parameter dimension  $\ell$ . `size` returns the matrix payload size, not the physical grid size.

# 4

## dpmat Reference

`dpmat` stores known finite real matrix data on a tensor grid. Objects can be coefficient-backed, function-only, or function-backed with explicit Bernstein coefficient evidence. Coefficient-backed objects participate in algebra. Function-only objects evaluate exactly through the stored handle, but do not expose coefficient evidence for `bernsteinTable` or coefficient algebra.

### 4.1 Lookup

| Task                 | Entry                                                                     |
|----------------------|---------------------------------------------------------------------------|
| Construct data       | <code>dpmat</code>                                                        |
| Evaluate             | <code>evaluate</code>                                                     |
| Inspect/display      | <code>bernsteinTable</code> , <code>disp</code> , <code>display</code>    |
| Plot                 | <code>plot</code>                                                         |
| Arithmetic           | <code>+</code> , <code>-</code> , unary <code>+/-</code> , <code>*</code> |
| Matrix/index methods | shape and structural methods, indexing and assignment                     |

### 4.2 Construction

**Syntax.**

```
A = dpmat(gridVector, source)
A = dpmat(gridVector, source, Degree=m)
A = dpmat(gridVectors, source)
A = dpmat(gridVectors, source, Degree=m)
```

**Arguments, options, and outputs.**

- `gridVector` is a numeric vector used as a one-parameter grid shorthand, such as `[0 1 2]`.
- `gridVectors` is a cell vector with one numeric grid vector per parameter, such as `\{[0 1], [10 20]\}`.
- `source` may be a function handle, a global cell grid of numeric Bernstein coefficients, or explicit nested `LocalValues`; for example, `\{1, 2, 3\}` is scalar degree-two data on one cell.
- `Degree=m` sets the local Bernstein degree. For function handles, omitting `Degree` creates a function-only object; passing a value such as `Degree=2` validates and stores Bernstein coefficient evidence while retaining the exact function handle for `evaluate`.

**Example.**

The scalar-grid shorthand is the shortest coefficient-backed form. The three numeric cells below are the global Bernstein coefficients for two physical cells with degree one.

```
A = dpmat([0 1 2], {10, 20, 30}, Degree=1)
size(A)
A.Degree
A.SourceSummary
```

```
A =
  dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     1

ans =
     1

ans =
  "coefficient-backed"
```

For tensor grids, pass one grid vector per parameter. The source cell array matches the global coefficient grid implied by the two parameter directions.

```
B = dpmat({[0 1], [10 20]}, {1 2; 3 4}, Degree=1)
B.GridInfo.Vectors{2}
B.lbls()
```

```
B =
  dpmat [1 1] over 2-D grid, degree 1, source coefficient-backed

ans =
    10    20

ans =
     0     0
     0     1
     1     0
     1     1
```

Use explicit nested `LocalValues` when each physical cell must be written directly. Here each cell stores two  $1 \times 2$  row-vector coefficients.

```
localVals = {[[1 0], [0 1]], {[2 0], [0 2]}};
C = dpmat({[0 1 2]}, localVals, Degree=1)
size(C)
C.coeffs(2)
```

```
C =
  dpmat [1 2] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     2

ans =
  1x2 cell array
    {[2 0]}    {[0 2]}
```

When `source` is a function handle and `Degree` is omitted, the object keeps the exact evaluator but does not claim coefficient evidence.

```
F = dpmat({[0 1]}, @(rho) [rho rho^2])
F.SourceSummary
F.evaluate(0.5)
```

```
F =
  dpmat [1 2] over 1-D grid, degree 1, source function

ans =
  "function"

ans =
  0.5000    0.2500
```

Add `Degree` for a polynomial function when later coefficient inspection or algebra should use Bernstein evidence. The exact function handle is still used for point evaluation.

```
G = dpmat({[0 1]}, @(rho) rho.^2, Degree=2)
G.SourceSummary
G.coeffs(1)
```

```
G =
  dpmat [1 1] over 1-D grid, degree 2, source function-bernstein

ans =
  "function-bernstein"

ans =
  1x3 cell array
  {[0]}    {[0]}    {[1]}
```

These forms all create finite known-data objects, but only the coefficient- backed and function- Bernstein forms can participate in coefficient algebra.

### 4.3 evaluate

#### Syntax.

```
val = evaluate(A, point)
val = A.evaluate(point)
```

#### Arguments, options, and outputs.

`point` is a scalar for a one-parameter object, such as 0.25, or a row vector with one entry per parameter, such as [0.25 12]. The return value is a numeric matrix with size `size(A)`. Coefficient-backed objects use local Bernstein interpolation. Function-backed objects evaluate through their stored function handle.

For a degree-one scalar coefficient-backed object on  $[\rho_0, \rho_1]$ ,

$$A(\rho) = aA_0 + (1 - a)A_1, \quad a = \frac{\rho_1 - \rho}{\rho_1 - \rho_0}.$$

The same local-coordinate rule is applied entrywise for matrix-valued data.

#### Example.

Use the function-call form when the object is naturally one input among several MATLAB expressions.

```
A = dpmat({[0 1]}, {2, 4}, Degree=1);
val = evaluate(A, 0.25)
```

```
val =
  2.5000
```

At  $\rho = 0.25$ , the local weight is  $a = 0.75$ , so the interpolated value is  $0.75 \cdot 2 + 0.25 \cdot 4 = 2.5$ .  
The dot-call form is equivalent and can be convenient in command-window exploration.

```
A = dpmat({[0 1]}, {2, 4}, Degree=1);
val = A.evaluate(0.75)
```

```
val =
  3.5000
```

Function-backed objects route through the retained function handle, so non-polynomial expressions can still be evaluated exactly at a point.

```
F = dpmat({[0 pi]}, @(rho) sin(rho));
val = F.evaluate(pi/2)
```

```
val =
  1
```

## 4.4 Visualization And Inspection

### 4.4.1 bernsteinTable

**Syntax.**

```
T = bernsteinTable(A)
T = bernsteinTable(A, cellSubs)
T = bernsteinTable(A, "oneLine")
T = bernsteinTable(A, cellSubs, "oneLine")
```

**Arguments, options, and outputs.**

- `cellSubs` selects one physical cell.
- `"oneLine"` returns one row per selected physical cell with a compact Bernstein expression.
- Without `"oneLine"`, the full table uses seven metadata columns; the output below shows their exact MATLAB names.

For one parameter, the **Basis** column represents

$$\binom{m}{j} a^{m-j} (1-a)^j.$$

For multiple parameters, the basis is the tensor product of that expression in each local coordinate. **LocalIndex** is the local Bernstein label. **CoeffSubscript** maps the local label back to the global coefficient subscript on the physical grid. **IsPhysicalNode** is true when that coefficient lies on a physical grid node.

**Example.**

```
A = dpmat({[0 1]}, {[0 1], [1 2]}, Degree=1);
T = bernsteinTable(A, 1, "oneLine");
disp(T)
```

| CellSubscript | Expression              |
|---------------|-------------------------|
| -----         | -----                   |
| {[1]}         | "a*[0 1] + (1-a)*[1 2]" |

Use the full table when the metadata columns are the point of the example:

```
A = dpmat({[0 1]}, {1, 2, 3}, Degree=2);
T = bernsteinTable(A);
disp(T)
```

| TermIndex | CellSubscript | CoeffSubscript | LocalIndex | Basis     | IsPhysicalNode | Value |
|-----------|---------------|----------------|------------|-----------|----------------|-------|
| -----     | -----         | -----          | -----      | -----     | -----          | ----- |
| 1         | {[1]}         | {[1]}          | {[0]}      | "a^2"     | true           | {[1]} |
| 2         | {[1]}         | {[2]}          | {[1]}      | "2a(1-a)" | false          | {[2]} |
| 3         | {[1]}         | {[3]}          | {[2]}      | "(1-a)^2" | true           | {[3]} |

#### 4.4.2 disp And display

##### Syntax.

```
disp(A)
display(A)
A
```

##### Arguments, options, and outputs.

`disp(A)` prints a compact one-line summary. `display(A)` is the command-window display used when an expression is not terminated by a semicolon; it prints grid, degree, coefficient, and source metadata.

##### Example.

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
disp(A)
display(A)
```

```
dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed
A =
dpmat with 1-by-1 matrix values
Parameters: 1
  rho_1: [0, 1], 2 nodes
Degree: 1
Physical cells: 1
Coefficients per cell: 2
Continuous: true
Source: coefficient-backed
Function handle: false
```

### 4.4.3 plot

#### Syntax.

```
h = plot(A)
h = plot(A, dims)
h = plot(A, SamplesPerCell=n)
h = plot(A, dims, SamplesPerCell=n)
h = plot(A, ..., Name, Value)
```

#### Arguments, options, and outputs.

- `dims` is one or two parameter indices. For one-parameter objects, the default is 1; otherwise the default is `[1 2]`, as in `plot(A, [1 2])`.
- `SamplesPerCell` is the package-owned sampling option. The default is 15; use `SamplesPerCell=4` for a coarse diagnostic plot.
- Remaining name-value pairs pass through to MATLAB graphics. One-dimensional plots pass them to `plot`; two-dimensional plots pass them to `surf`. Examples include `LineWidth`, `FaceAlpha`, and `EdgeColor`.
- The output `h` is the vector of graphics handles, one entry per matrix element.

#### Example.

The one-dimensional call returns one graphics handle per matrix entry. The example inspects the generated sample count and line styling without relying on an interactive figure window.

```
fig = figure('Visible','off');
A = dpmat({[0 1]}, @(rho) [rho, rho^2]);
h = plot(A, SamplesPerCell=4, LineWidth=2);
numel(h)
numel(h(1).XData)
h(1).LineWidth
h(1).YData([1 end])
close(fig)
```

```
ans =
     2

ans =
     5

ans =
     2

ans =
     0     1
```

With four samples per cell on one physical interval, MATLAB stores five plotted node values including both endpoints.

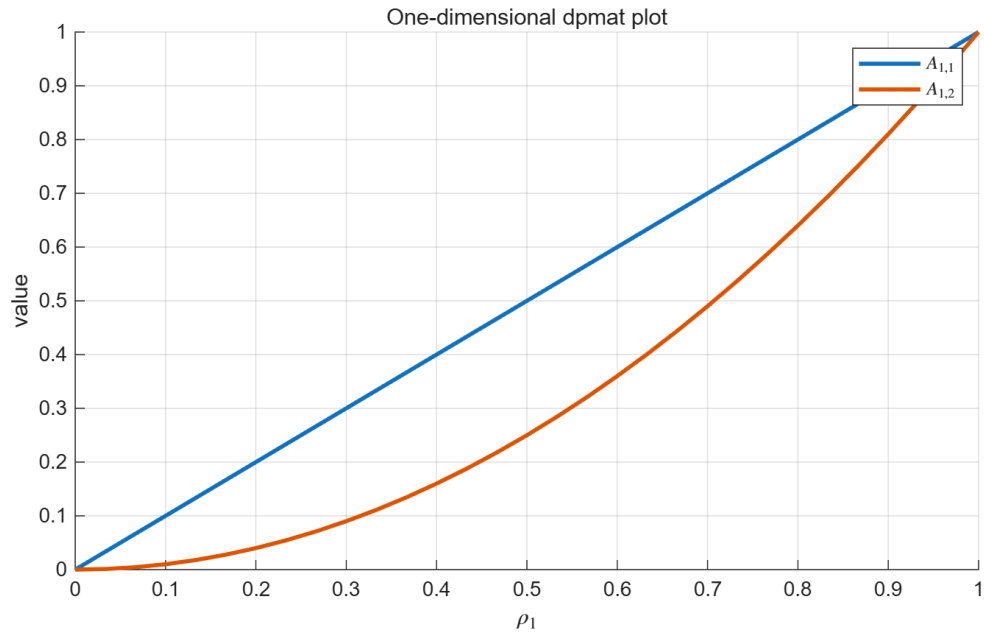


Figure 2: One-dimensional `dpmat/plot` output generated from MATLAB.

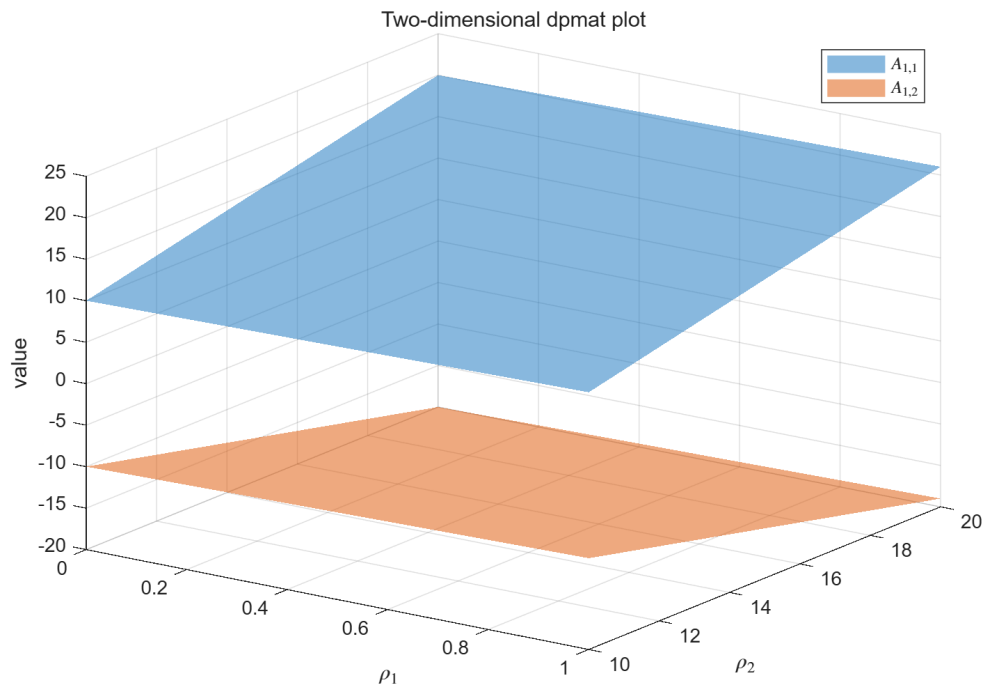


Figure 3: Two-dimensional `dpmat/plot` surface output generated from MATLAB.

## 4.5 Algebra Operators

### Syntax.

```
C = A + B
C = A + M
C = M + A
C = A - B
C = A - M
C = M - A
C = +A
C = -A
C = A * B
C = A * M
C = M * A
```

### Arguments, options, and outputs.

- Coefficient-backed **dpmat**: addition, subtraction, unary signs, and multiplication use local Bernstein coefficients.
- Numeric scalar, such as 5, broadcasts to the **dpmat** matrix size when needed.
- Numeric matrix: accepted when the matrix size is compatible with the operation, such as **eye(2)** for a  $2 \times 2$  object.
- Mixed grids with same bounds: re-expressed on a common refinement.
- Mixed degrees: elevated to a common degree for coefficient-wise operations; multiplication produces the degree sum.
- Function-only **dpmat**: evaluates and plots, but does not enter coefficient algebra unless explicit **Degree** supplied coefficient evidence.

For addition and subtraction, the package elevates both operands to a common degree before combining matching coefficients:

$$C_j = \hat{A}_j + \hat{B}_j.$$

For multiplication, coefficient labels convolve and the output degree is the sum of the input degrees.

### Example.

Addition elevates the lower-degree operand before combining coefficients.

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
B = dpmat({[0 1]}, {10, 20, 30}, Degree=2);
S = A + B
cS = S.coeffs(1);
[cS{:}]
```

```
S =
  dpmat [1 1] over 1-D grid, degree 2, source coefficient-backed

ans =
    11.0000    21.5000    32.0000
```

Multiplication raises degree from 1 and 2 to 3.

```
A = dpmat([0 1]), {1, 2}, Degree=1);
B = dpmat([0 1]), {10, 20, 30}, Degree=2);
P = A * B
cP = P.coeffs(1);
[cP{:}]
```

```
P =
  dpmat [1 1] over 1-D grid, degree 3, source coefficient-backed

ans =
    10.0000    20.0000    36.6667    60.0000
```

Numeric scalars are promoted to compatible constant coefficient data.

```
A = dpmat([0 1]), {1, 2}, Degree=1);
M = 5 - A
N = 2 * A
cM = M.coeffs(1);
cN = N.coeffs(1);
[cM{:}]
[cN{:}]
```

```
M =
  dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed

N =
  dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     4     3

ans =
     2     4
```

These examples use the one-cell case so the coefficient vectors are visible; the same elevation and convolution rules apply cell by cell on tensor grids.

## 4.6 Matrix Methods

**Syntax.**

```
sz = size(A)
n = length(A)
n = height(A)
n = width(A)
n = numel(A)
n = ndims(A)

B = A.'
B = A'
B = vec(A)
B = diag(A)
B = diag(A, k)
B = reshape(A, sz)
B = reshape(A, m, n)
```

```

B = tril(A)
B = triu(A)
B = trace(A)
B = sum(A)
B = sum(A, dim)
B = sum(A, "all")
B = mean(A)
B = mean(A, dim)
B = mean(A, "all")
B = cumsum(A)
B = cumsum(A, dim)
B = flip(A)
B = flip(A, dim)
B = flipud(A)
B = fliplr(A)
B = rot90(A)
B = rot90(A, k)
B = repmat(A, ...)
B = [A, B]
B = [A; B]
B = cat(dim, A, B, ...)
B = blkdiag(A, B, ...)
tf = isequal(A, B, ...)

```

### Arguments, options, and outputs.

These methods apply the corresponding MATLAB matrix operation to every local coefficient. They require coefficient evidence. Dimension arguments follow MATLAB conventions, such as `sum(A,2)` for row sums or `reshape(A, [1 4])` for a fixed output shape. Concatenation and `blkdiag` accept compatible coefficient-backed `dpmat` objects and numeric blocks, align grids, and elevate degrees when needed.

### Example.

Structural methods return new `dpmat` objects whose matrix size changes while the grid and degree metadata stay attached to the coefficient storage.

```

A = dpmat({[0 1]}, {[1 2; 3 4]}, [10 20; 30 40]), Degree=1);
V = vec(A)
size(V)
D = diag(A)
size(D)
R = reshape(A, [1 4])
size(R)

```

```

V =
  dpmat [4 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     4     1

D =
  dpmat [2 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     2     1

```

```
R =
    dpmat [1 4] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     4
```

Summary and assembly methods follow the same coefficient-wise rule.

```
A = dpmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
Tr = trace(A)
H = [diag(A), diag(A)]
size(Tr)
size(H)
```

```
Tr =
    dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed

H =
    dpmat [2 2] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     1

ans =
     2     2
```

## 4.7 Indexing And Assignment

### Syntax.

```
B = A(rows, cols)
value = A.PropertyName
A(rows, cols) = numericValue
A(rows, cols) = B
```

### Arguments, options, and outputs.

Matrix indexing slices every local coefficient. Dot access exposes public properties and methods. Assignment accepts numeric blocks and compatible coefficient-backed **dpmat** blocks, with degree elevation when needed. Use ordinary MATLAB matrix subscripts such as `A(:, end)` or `A(1, 2:3)`.

### Example.

Indexing returns a smaller **dpmat**; assignment updates the corresponding matrix entries in every local coefficient.

```
A = dpmat({[0 1]}, {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
lastCol = A(:, end)
topTail = A(1, 2:3)
A(:, 2) = 5;
A
```

```
lastCol =
    dpmat [2 1] over 1-D grid, degree 1, source coefficient-backed

topTail =
    dpmat [1 2] over 1-D grid, degree 1, source coefficient-backed

A =
    dpmat [2 3] over 1-D grid, degree 1, source coefficient-backed
```

The assignment does not change the object degree because the numeric value is promoted to degree-one coefficient data before insertion.

## 4.8 Limits

- Function-only objects created without `Degree` do not have Bernstein coefficient evidence for `bernsteinTable` or coefficient algebra.
- Numeric operands must be finite real nonempty matrices with compatible sizes, or finite real scalars that can broadcast to the required size.
- Grid refinement is supported for matching parameter dimensions and matching grid bounds.

# 5

## dpvar Reference

`dpvar` creates continuous cell-local Bernstein decision expressions backed by YALMIP `sdpvar` coefficients. It participates in supported affine and known-data algebra. It does not choose solvers or call `optimize`; solver-facing work happens after comparison overloads create `dplmi` objects and `toYalmip` converts them to YALMIP constraints.

### 5.1 Lookup

| Task                 | Entry                                                                               |
|----------------------|-------------------------------------------------------------------------------------|
| Construct decisions  | <code>dpvar</code>                                                                  |
| Differentiate rates  | <code>rhodiff</code>                                                                |
| Affine/mixed algebra | <code>+</code> , <code>-</code> , <code>*</code> with known data                    |
| Matrix/index methods | <code>structural methods</code> , <code>indexing</code> and <code>assignment</code> |
| Build LMIs           | <code>&lt;=</code> , <code>&gt;=</code>                                             |

### 5.2 Construction

**Syntax.**

```
P = dpvar(n, gridVectors)
P = dpvar(n, n, gridVectors)
P = dpvar(n, m, gridVectors)
P = dpvar(..., "full")
P = dpvar(..., "symmetric")
P = dpvar(..., Degree=0)
P = dpvar(..., Degree=1)
P = dpvar(..., RateBounds=rb)
```

**Arguments, options, and outputs.**

- `dpvar(n, gridVectors)` creates an  $n \times n$  square variable using scalar-size shorthand; for example, `dpvar(2, \{[0 1]\})`. Square variables default to symmetric YALMIP coefficients.
- `dpvar(n, n, gridVectors)` is the explicit square-size form; for example, `dpvar(2, 2, \{[0 1]\})`. It has the same default symmetric structure as the shorthand.
- `dpvar(n, m, gridVectors)` creates an  $n \times m$  variable. Rectangular variables default to full YALMIP coefficients; for example, `dpvar(2, 3, \{[0 1]\})`.
- "full" and "symmetric" explicitly choose the YALMIP coefficient structure. "symmetric" requires a square size.
- `Degree=1` is the default continuous Bernstein decision variable.
- `Degree=0` creates a parameter-independent variable stored on the grid.

- `RateBounds=rb` stores parameter-rate metadata for later `rhodiff(P)` calls. Use `RateBounds=[-1 1]` for one parameter; use a two-row matrix for two parameters, as shown in the rate-metadata example below.

### Example.

The square shorthand creates a  $2 \times 2$  continuous decision object. The default degree is one and the object carries no rate metadata.

```
yalmip('clear')
Ps = dpvar(2, {[0 1]});
class(Ps)
size(Ps)
Ps.Degree
Ps.HasRateDependence
```

```
ans =
    'dpvar'

ans =
     2     2

ans =
     1

ans =
    logical
     0
```

Use two dimension arguments for a rectangular full variable.

```
yalmip('clear')
Pf = dpvar(2, 3, {[0 1]});
class(Pf)
size(Pf)
Pf.Degree
```

```
ans =
    'dpvar'

ans =
     2     3

ans =
     1
```

The structure flag makes the coefficient structure explicit. This is useful when a square variable should use full, not symmetric, coefficients.

```
yalmip('clear')
Pfull = dpvar(2, 2, {[0 1]}, "full");
class(Pfull)
Pfull.MatrixSize
Pfull.Degree
```

```
ans =
    'dpvar'

ans =
```

```

      2      2
ans =
      1

```

Use `Degree=0` for a parameter-independent decision variable stored with grid metadata.

```

yalmip('clear')
P0 = dpvar(1, [0 1 2], Degree=0);
class(P0)
P0.Degree
P0.ncell()

```

```

ans =
    'dpvar'

ans =
      0

ans =
      2

```

Rate bounds are metadata for later `rhodiff(P)` calls. `dplmi` constrains rate vertices only after an expression such as `rhodiff` has stored rate-vertex rows.

```

yalmip('clear')
Pr = dpvar(1, {[0 1], [10 20]}, RateBounds=[-1 2; -3 4]);
class(Pr)
Pr.HasRateDependence
Pr.RateBounds

```

```

ans =
    'dpvar'

ans =
    logical
      1

ans =
    -1      2
    -3      4

```

### 5.3 rhodiff

#### Syntax.

```

D = rhodiff(P)
D = rhodiff(P, rb)

```

#### Arguments, options, and outputs.

`rhodiff(P)` uses `P.RateBounds`; `rhodiff(P, rb)` supplies rate bounds at the call site, such as `rhodiff(P, [-1 1])`. The output is a discontinuous `dpvar` expression with one coefficient row per rate vertex. In one parameter, a degree- $m$  derivative has degree  $m - 1$ , with degree zero retained as a zero derivative. In multiple parameters, partial derivatives are elevated back into the common tensor degree used by the implementation.

For a one-parameter degree-one coefficient pair  $P_0, P_1$  on a cell of width  $h$ , the rate-dependent derivative row is

$$\dot{P}(\rho, \dot{\rho}) = \dot{\rho} \frac{P_1 - P_0}{h}.$$

The implementation stores one row for each allowed rate vertex.

### Example.

When rate bounds live on the object, `rhodiff(P)` uses them directly.

```
yalmip('clear')
P = dpvar(2, {[0 1 2]}, "symmetric", RateBounds=[-1 1]);
D = rhodiff(P);
class(D)
D.Degree
D.IsContinuous
size(D.coeffs(1))
D.RateBounds
```

```
ans =
    'dpvar'

ans =
    0

ans =
    logical
    0

ans =
     2     1

ans =
    -1     1
```

For a two-parameter object, the rate-vertex rows enumerate all lower/upper combinations from the two rows of `RateBounds`.

```
yalmip('clear')
Q = dpvar(1, {[0 1], [10 20]}, RateBounds=[-1 1; -2 2]);
E = rhodiff(Q);
class(E)
E.Degree
size(E.coeffs([1 1]))
E.RateBounds
```

```
ans =
    'dpvar'

ans =
    1

ans =
     4     4

ans =
    -1     1
    -2     2
```

## 5.4 Affine And Mixed Algebra

### Syntax.

```
R = P + Q
R = P + A
R = A + P
R = P + M
R = M + P
R = P - Q
R = P - A
R = A - P
R = +P
R = -P
R = A * P
R = P * A
R = M * P
R = P * M
```

### Arguments, options, and outputs.

- **dpvar**: affine addition and subtraction with compatible size, grid, and rate metadata.
- Coefficient-backed **dpmat**: mixed known/decision algebra for ordinary **dpvar** expressions, including products where only one side contains decisions.
- Numeric scalar or matrix: promoted as constant coefficient data for ordinary **dpvar** expressions when finite real and size-compatible.
- Affine **sdpmat** matrix: promoted as parameter-independent affine data when real, linear, and size-compatible.
- Rate-vertex derivative rows: supported in the tested derivative and known-data combinations used by **rhodiff** LMI residuals.
- Metadata-only rate-dependent **dpvar** products are unsupported until **rhodiff** has materialized rate-vertex rows.

### Example.

The affine sum with a numeric matrix keeps the same grid and degree.

```
yalmp('clear')
P = dpvar(2, {[0 1]}, "symmetric");
S = P + eye(2);
class(S)
size(S)
S.Degree
S.ContainsDecision
```

```
ans =
    'dpvar'
ans =
```

```

      2      2
ans =
      1

ans =
    logical
      1

```

Known-data products are allowed when only one side contains decisions. The product degree is the sum of the known-data degree and the decision degree.

```

yal mip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
A = dpmat([0 1], {eye(2), 2*eye(2)}, Degree=1);
K = A * P;
R = P * A;
class(K)
class(R)
K.Degree
R.Degree

```

```

ans =
    'dpvar'

ans =
    'dpvar'

ans =
      2

ans =
      2

```

## 5.5 Matrix Methods

### Syntax.

```

sz = size(P)
n = length(P)
n = height(P)
n = width(P)
n = numel(P)
n = ndims(P)

Q = P.'
Q = P'
Q = vec(P)
Q = diag(P)
Q = diag(P, k)
Q = reshape(P, sz)
Q = reshape(P, m, n)
Q = tril(P)
Q = triu(P)
Q = trace(P)
Q = sum(P)
Q = sum(P, dim)

```

```

Q = sum(P, "all")
Q = mean(P)
Q = mean(P, dim)
Q = mean(P, "all")
Q = cumsum(P)
Q = cumsum(P, dim)
Q = flip(P)
Q = flip(P, dim)
Q = flipud(P)
Q = fliplr(P)
Q = rot90(P)
Q = rot90(P, k)
Q = repmat(P, ...)
Q = [P, R]
Q = [P; R]
Q = cat(dim, P, R, ...)
Q = blkdiag(P, R, ...)
tf = isequal(P, R, ...)

```

### Arguments, options, and outputs.

These methods preserve Bernstein grid metadata and act coefficient-wise on the matrix payloads. Dimension arguments follow MATLAB-style calls such as `sum(P,2)`, `reshape(P,[3 2])`, and `diag(P,1)`. Concatenation and `blkdiag` may combine `dpvar`, coefficient-backed `dpmat`, affine `sdpvar`, and numeric blocks when the matrix dimensions, grid bounds, degree elevation, and rate metadata are compatible.

### Example.

Shape-changing operations return new `dpvar` objects and preserve the decision-backed coefficient graph.

```

yal mip('clear')
P = dpvar(2, 3, {[0 1]}, "full");
Tt = P.';
class(Tt)
size(Tt)
V = vec(P);
class(V)
size(V)
R = reshape(P, [3 2]);
class(R)
size(R)

```

```

ans =
    'dpvar'

ans =
     3     2

ans =
    'dpvar'

ans =
     6     1

ans =
    'dpvar'

```

```
ans =
     3     2
```

Summary and assembly calls use the same coefficient-wise mechanism.

```
yalmip('clear')
P = dpvar(2, 3, {[0 1]}, "full");
S = sum(P, 2);
B = blkdiag(P(1:2,1:2), eye(2));
class(S)
class(B)
size(S)
size(B)
```

```
ans =
     'dpvar'

ans =
     'dpvar'

ans =
     2     1

ans =
     4     4
```

## 5.6 Indexing And Assignment

**Syntax.**

```
Q = P(rows, cols)
value = P.PropertyName
P(rows, cols) = numericValue
P(rows, cols) = Q
P(rows, cols) = A
P(rows, cols) = X
```

**Arguments, options, and outputs.**

Matrix indexing slices the matrix payload across every physical cell and local coefficient. Assignment accepts numeric blocks, compatible **dpvar** blocks, compatible coefficient-backed **dpmat** blocks, and affine **sdpvar** blocks. Assignment may elevate degrees and propagate compatible rate metadata. Use ordinary MATLAB subscripts such as  $P(1:2,1)$  or  $P(:,2)=A$ .

**Example.**

Indexing returns a smaller **dpvar**; assignment updates the selected matrix entries across every local coefficient.

```
yalmip('clear')
P = dpvar(2, 3, {[0 1]}, "full");
col = P(:, 2);
P(:, 1) = 0;
class(col)
size(col)
size(P)
```

```
ans =
    'dpvar'

ans =
     2     1

ans =
     2     3
```

## 5.7 Comparisons To dplmi

### Syntax.

```
C = P <= 0
C = P >= 0
C = lhs <= rhs
C = lhs >= rhs
```

### Arguments, options, and outputs.

The comparison overloads construct `dplmi` objects. The expression must be square and coefficient-wise symmetric or Hermitian after assembly. `rhs` is usually 0, but compatible numeric, `dpmat`, affine `sdpvar`, and `dpvar` expressions may appear in the residual. `dpvar` comparisons do not solve an optimization problem by themselves.

### Example.

Comparison creates a `dplmi` wrapper with one stored constraint for each physical cell and local Bernstein coefficient.

```
yal mip('clear')
P = dpvar(2, {[0 0.5 1]}, "symmetric");
Cneg = P <= 0;
class(Cneg)
numel(Cneg.Constraints)
Cpos = P >= 0;
class(Cpos)
numel(Cpos.Constraints)
```

```
ans =
    'dplmi'

ans =
     4

ans =
    'dplmi'

ans =
     4
```

## 5.8 Limits

- Constructor-created `dpvar` objects currently support `Degree=0` and `Degree=1`.
- Products may contain decision variables on at most one side; nonlinear decision products such as  $P * Q$  are outside this slice.

- Function-only `dpmat` operands cannot enter `dpvar` algebra unless they were constructed with explicit Bernstein evidence.
- `rhodiff` of an existing rate-vertex derivative expression is not part of the current public workflow.

# 6

## dplmi Reference

`dplmi` stores direct coefficient-wise YALMIP constraints assembled from square symmetric `dpvar` expressions. It is the package boundary between Bernstein coefficient objects and ordinary YALMIP solver calls.

### 6.1 Lookup

| Task              | Entry                                                  |
|-------------------|--------------------------------------------------------|
| Construct wrapper | <code>dplmi</code> , <code>&lt;=/&gt;=</code>          |
| Count constraints | <code>direct coefficient constraints</code>            |
| Export to YALMIP  | <code>toYalmip</code>                                  |
| Solve             | <code>solver-facing use</code> , <code>Examples</code> |

### 6.2 Construction And Comparisons

#### Syntax.

```
C = dplmi(expr, relation)
C = dplmi(expr, relation, relaxLemma=false, UsePolya=false, PolyDegree=0)
C = lhs <= rhs
C = lhs >= rhs
```

#### Arguments, options, and outputs.

`expr` must be a square `dpvar` expression whose coefficient matrices are symmetric or Hermitian. The `relation` input is "`<=`" or "`>=`". Comparisons such as `lhs >= rhs` are the ordinary user-facing entry point and support compatible numeric, `dpmat`, affine `sdpvar`, and `dpvar` right-hand sides. Direct `dplmi(expr, relation)` construction is useful for advanced assembly or tests. Reserved options must stay at their defaults: `relaxLemma=false`, `UsePolya=false`, and `PolyaDegree=0`; nondefault values currently raise reserved-option errors.

#### Example.

The direct constructor is equivalent to the comparison path when the relation and reserved options are explicit.

```
yalmip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
C = dplmi(P, ">=", relaxLemma=false, UsePolya=false, PolyDegree=0);
class(C)
numel(C.Constraints)
C.RelaxLemma
C.UsePolya
C.PolyaDegree
```

```

ans =
    'dplmi'

ans =
    2

ans =
    logical
    0

ans =
    logical
    0

ans =
    0

```

### 6.3 Direct Coefficient Assembly

For each physical cell, local Bernstein coefficient, and rate-vertex row if present, `dplmi` creates one YALMIP semidefinite constraint. For an ordinary expression with  $N_c$  physical cells and  $N_b$  Bernstein coefficients per cell, the number of stored constraints is  $N_c N_b$ . For a `rhodiff` expression with  $N_v$  rate vertices, the count becomes  $N_c N_b N_v$ .

### 6.4 toYalmip

**Syntax.**

```

F = toYalmip(C)
F = C.toYalmip()

```

**Arguments, options, and outputs.**

`toYalmip` concatenates the stored constraint cells into a YALMIP constraint array. It has no package-specific options. The output is suitable for `optimize`, concatenation with other YALMIP constraints, or ordinary YALMIP inspection.

**Example.**

The function-call and dot-call forms return equivalent YALMIP constraint arrays.

```

yalmip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
C = P >= 0;
class(C)
numel(C.Constraints)
F1 = toYalmip(C);
F2 = C.toYalmip();
isa(F1, "lmi") || isa(F1, "constraint")
length(F1)
isa(F2, "lmi") || isa(F2, "constraint")
length(F2)

```

```

ans =
    'dplmi'

ans =

```

```

2
ans =
    logical
     1
ans =
     2
ans =
    logical
     1
ans =
     2

```

After this boundary, the package is no longer assembling Bernstein data; YALMIP owns solver choice, diagnostics, and optimization status.

## 6.5 Solver-Facing Use

`dplmi` does not own solver settings or wrap `optimize`. After `toYalmip`, use ordinary YALMIP:

```

opts = sdpsettings("solver", solverName, "verbose", 0);
sol = optimize([C1.toYalmip, C2.toYalmip], objective, opts);

```

The solver selected by YALMIP must be available on the MATLAB path. The current test suite prefers MOSEK when available and otherwise falls back to LMILAB for smoke coverage.

# 7

## Examples

### 7.1 Rate-Dependent Block LMI Solver Smoke

This example is adapted from `+tests/+dplmi/test_solver_smoke.m`. The goal is to find a parameter-dependent  $P(\rho)$  and scalar  $\gamma$  such that, on every Bernstein coefficient and rate vertex,

$$\begin{bmatrix} \dot{P} + PA + A^\top P & PB & C^\top \\ B^\top P & -\gamma I & D^\top \\ C & D & -\gamma I \end{bmatrix} \preceq 0, \quad P(\rho) \succeq 0.$$

The derivative term  $\dot{P}$  is built with `rhodiff(P, [-1 1])`. The comparison operators create `dplmi` wrappers, and `toYalmip` hands the coefficient-wise constraints to YALMIP.

```
yalmip('clear')
grid = linspace(0, 1, 2);
A = dpmat(grid, @(x) [-1, 0.5; -1, -2] + ...
    x * [-1.3, -20; 2, -10], Degree=1);
B = dpmat(grid, @(x) [1, -4; -1, -1] + ...
    x * [2.2, 0.5; -6, -5], Degree=1);
Cmat = eye(2);
Dmat = zeros(2);

P = dpvar(2, grid);
diffP = rhodiff(P, [-1 1]);
gamma = dpvar(1, grid, Degree=0);
E1 = [diffP + P * A + A' * P, P * B, Cmat';
    B' * P, -gamma * eye(2), Dmat';
    Cmat, Dmat, -gamma * eye(2)] <= 0;
E2 = P >= 0;

objective = gamma.LocalValues{1}{1};
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);
sol = optimize([E1.toYalmip, E2.toYalmip], objective, opts);

numel(E1.Constraints)
numel(E2.Constraints)
sol.problem
isfinite(value(objective))
```

```
ans =
    6

ans =
    2
```

```
ans =  
    0  
  
ans =  
logical  
    1
```

# 8

## Current Limits

- `dplmi` currently assembles direct coefficient-wise constraints. Relaxation-lemma and Polyta workflows are reserved future layers, not supported user workflows in this manual.
- Package-owned solver wrappers, strictness-margin diagnostics, residual evidence, and advanced relaxation workflows are not part of the current public API.
- `dpvar` is an affine YALMIP/Bernstein expression object. It does not support nonlinear decision products.
- `dpmat` function-only objects evaluate exactly, but coefficient algebra requires Bernstein coefficient evidence.
- `dpbase` is documented for architecture and advanced inspection. Primary modeling workflows should use `dpmat`, `dpvar`, and `dplmi`.